CELEBAL INTERNSHIP

NAME – ANMOL BHATIA

Q1: Explain the roles of the **Driver**, **Cluster Manager**, and **Executor** in a Spark application.

In a Spark application, the Driver is responsible for controlling the entire application. It creates the Spark Session, prepares the execution plan, divides the work into smaller tasks, and sends those tasks to the executors. The Cluster Manager is responsible for providing resources like CPU and memory and deciding where the executors will run. Executors are the worker processes that perform the actual computation by processing different parts of the data and sending the results back to the Driver. In simple terms, the Driver manages the application, the Cluster Manager provides the required resources, and the Executors do the actual processing.

Q2: How does Spark's **Lazy Evaluation** strategy improve performance when chain-processing large datasets?

Spark uses lazy evaluation, which means it does not execute transformations like 'filter()' or 'select()' immediately. Instead, it keeps track of all the operations and waits until an action such as 'show()' or 'count()' is called. This allows Spark to optimize the entire execution plan before running it. As a result, unnecessary computations are avoided, filters can be applied earlier, and only the required data is processed. This makes Spark much faster and more efficient, especially when working with large datasets

Q3: Write a Spark command to read a CSV file located at `"data/source.csv"`, ensuring the first row is treated as a header and `inferSchema` is enabled.

```
[5]
✓ 2s
df.write \
    .mode("overwrite") \
    .option("header", True) \
    .csv("/content/data/source.csv")

[6]
✓ 3s
csv_df = spark.read.csv(
    "/content/data/source.csv",
    header=True,
    inferSchema=True
)

csv_df.show()
csv_df.printSchema()

+----------+-----------+------------+---------+------+--------+------+----------+-------+
|product_id|   category|product_name|   status|region|priority|amount|base_price|user_id|
+----------+-----------+------------+---------+------+--------+------+----------+-------+
|       103|Electronics|       Phone|Completed|  East|    High|1500.0|    1300.0| user_3|
|       104|  Furniture|       Chair|Cancelled|  West|     Low| 500.0|     450.0|   NULL|
|       105|Electronics|      Tablet|Completed| North|  Medium|1100.0|     950.0| user_5|
|       101|Electronics|      Laptop|Completed| North|    High|1200.0|    1000.0| user_1|
|       102|   Clothing|      Jacket|  Pending| South|  Medium| 800.0|     700.0| user_2|
+----------+-----------+------------+---------+------+--------+------+----------+-------+

root
 |-- product_id: integer (nullable = true)
 |-- category: string (nullable = true)
 |-- product_name: string (nullable = true)
 |-- status: string (nullable = true)
 |-- region: string (nullable = true)
 |-- priority: string (nullable = true)
 |-- amount: double (nullable = true)
 |-- base_price: double (nullable = true)
 |-- user_id: string (nullable = true)
```

Q4: What is the difference between **CSV** and **Parquet** in terms of storage (row-based vs. columnar) and why does it matter for performance?

CSV is a row-based file format, which means it stores all the data of one row together. It is simple to use and can be opened in almost any spreadsheet application, but it takes up more space and is slower when working with large datasets. On the other hand, Parquet is a column-based file format that stores data column by column. Since Spark often reads only a few columns during analysis, Parquet can load only the required data instead of scanning the entire file. It also supports compression and stores the schema, making it much faster and more efficient for big data processing.

Q5: Given a DataFrame `df`, write a query to select the columns `product_id` and `price` where the `category` is 'Electronics'.

```
[11]
✓ 0s
```

```python
q5_result = df.filter(df["category"] == "Electronics").select(
    "product_id",
    "base_price"
    )

q5_result.show()
```

```
+----------+----------+
|product_id|base_price|
+----------+----------+
|       101|    1000.0|
|       103|    1300.0|
|       105|     950.0|
+----------+----------+
```

Q6: Write the code to "revise" a DataFrame by renaming the column `old_name` to `new_name` and casting the `price` column from a String to a Double.

```python
from pyspark.sql.functions import col

string_df = df.withColumn(
    "base_price",
    col("base_price").cast("string")
)

q6_result = string_df \
    .withColumnRenamed("product_name", "product") \
    .withColumn(
        "base_price",
        col("base_price").cast("double")
    )

q6_result.show()
q6_result.printSchema()
```

```
+----------+-----------+-------+---------+------+--------+------+----------+-------+
|product_id|   category|product|   status|region|priority|amount|base_price|user_id|
+----------+-----------+-------+---------+------+--------+------+----------+-------+
|       101|Electronics| Laptop|Completed| North|    High|1200.0|    1000.0| user_1|
|       102|   Clothing| Jacket|  Pending| South|  Medium| 800.0|     700.0| user_2|
|       103|Electronics|  Phone|Completed|  East|    High|1500.0|    1300.0| user_3|
|       104|  Furniture|  Chair|Cancelled|  West|     Low| 500.0|     450.0|   NULL|
|       105|Electronics| Tablet|Completed| North|  Medium|1100.0|     950.0| user_5|
+----------+-----------+-------+---------+------+--------+------+----------+-------+

root
 |-- product_id: long (nullable = true)
 |-- category: string (nullable = true)
 |-- product: string (nullable = true)
 |-- status: string (nullable = true)
 |-- region: string (nullable = true)
 |-- priority: string (nullable = true)
 |-- amount: double (nullable = true)
 |-- base_price: double (nullable = true)
 |-- user_id: string (nullable = true)
```

Q7: How does Spark use the **Lineage Graph (DAG)** to provide fault tolerance if a worker node fails?

Spark keeps track of every transformation performed on a Data Frame by creating a Lineage Graph, also known as a DAG (Directed Acyclic Graph). If a worker node or executor fails, Spark doesn't have to restart the entire application. Instead, it uses the DAG to identify the missing data and recomputes only the lost partitions using the same sequence of transformations. This makes Spark fault tolerant and helps recover from failures efficiently without affecting the rest of the processing.

Q8: Write a query to filter a DataFrame `df_orders` for rows where the `status` is 'Completed' AND the `amount` is greater than 1000.

```
q8_result = df.filter(
    (col("status") == "Completed") &
    (col("amount") > 1000)
)

q8_result.show()
```

```
+----------+-----------+------------+---------+------+--------+------+----------+-------+
|product_id|   category|product_name|   status|region|priority|amount|base_price|user_id|
+----------+-----------+------------+---------+------+--------+------+----------+-------+
|       101|Electronics|      Laptop|Completed| North|    High|1200.0|    1000.0| user_1|
|       103|Electronics|       Phone|Completed|  East|    High|1500.0|    1300.0| user_3|
|       105|Electronics|      Tablet|Completed| North|  Medium|1100.0|     950.0| user_5|
+----------+-----------+------------+---------+------+--------+------+----------+-------+
```

Q9: Explain the concept of **Predicate Pushdown** in Parquet and how it affects the amount of data loaded into memory.

Predicate Pushdown is a feature of Parquet that allows Spark to apply filter conditions while reading the data instead of after loading the entire file. This means Spark reads only the rows or columns that match the filter condition, instead of scanning the complete dataset. As a result, less data is loaded into memory, disk reads are reduced, and queries run faster. This is one of the main reasons why Parquet performs much better than CSV for large analytical workloads.

Q10: Write a code snippet to add a new column `final_price` which is the `base_price` multiplied by 1.18 (18% tax).

```
q10_result = df.withColumn("final_price",col("base_price") * 1.18)

q10_result.show()
```

```
+----------+-----------+------------+---------+------+--------+------+----------+-------+-----------+
|product_id|   category|product_name|   status|region|priority|amount|base_price|user_id|final_price|
+----------+-----------+------------+---------+------+--------+------+----------+-------+-----------+
|       101|Electronics|      Laptop|Completed| North|    High|1200.0|    1000.0| user_1|     1180.0|
|       102|   Clothing|      Jacket|  Pending| South|  Medium| 800.0|     700.0| user_2|      826.0|
|       103|Electronics|       Phone|Completed|  East|    High|1500.0|    1300.0| user_3|     1534.0|
|       104|  Furniture|       Chair|Cancelled|  West|     Low| 500.0|     450.0|   NULL|      531.0|
|       105|Electronics|      Tablet|Completed| North|  Medium|1100.0|     950.0| user_5|     1121.0|
+----------+-----------+------------+---------+------+--------+------+----------+-------+-----------+
```

Q11: What is the difference between **Transformations** and **Actions**? Provide two examples of each.

Transformations are operations that create a new DataFrame but do not execute immediately. Spark only records them in the execution plan. Examples include ‘filter()’ and ‘select()’.

Actions are operations that actually trigger Spark to run the execution plan and return or save the result. Examples include ‘show()’ and ‘count()’.

```
filtered_df = df.filter(df["amount"] > 1000)
selected_df = df.select("product_id", "category")

filtered_df.show()
print("Total rows:", selected_df.count())
```

```
+----------+-----------+------------+---------+------+--------+------+----------+-------+
|product_id|   category|product_name|   status|region|priority|amount|base_price|user_id|
+----------+-----------+------------+---------+------+--------+------+----------+-------+
|       101|Electronics|      Laptop|Completed| North|    High|1200.0|    1000.0| user_1|
|       103|Electronics|       Phone|Completed|  East|    High|1500.0|    1300.0| user_3|
|       105|Electronics|      Tablet|Completed| North|  Medium|1100.0|     950.0| user_5|
+----------+-----------+------------+---------+------+--------+------+----------+-------+

Total rows: 5
```

Q12: Write the Spark command to load a Parquet file from `"path/to/input"`, filter out any rows where `user_id` is null, and save the result as a CSV at `"path/to/output"`.

```
df.write \
    .mode("overwrite") \
    .parquet("/content/data/source_parquet")

input_df = spark.read.parquet("/content/data/source_parquet")

filtered_df = input_df.filter(col("user_id").isNotNull())

filtered_df.show()

filtered_df.write \
    .mode("overwrite") \
    .option("header", True) \
    .csv("/content/output")

output_df = spark.read.csv("/content/output",header=True,inferSchema=True)

output_df.show()
```

```
+----------+-----------+------------+---------+------+--------+------+----------+-------+
|product_id|   category|product_name|   status|region|priority|amount|base_price|user_id|
+----------+-----------+------------+---------+------+--------+------+----------+-------+
|       103|Electronics|       Phone|Completed|  East|    High|1500.0|    1300.0| user_3|
|       105|Electronics|      Tablet|Completed| North|  Medium|1100.0|     950.0| user_5|
|       101|Electronics|      Laptop|Completed| North|    High|1200.0|    1000.0| user_1|
|       102|   Clothing|      Jacket|  Pending| South|  Medium| 800.0|     700.0| user_2|
+----------+-----------+------------+---------+------+--------+------+----------+-------+

+----------+-----------+------------+---------+------+--------+------+----------+-------+
|product_id|   category|product_name|   status|region|priority|amount|base_price|user_id|
+----------+-----------+------------+---------+------+--------+------+----------+-------+
|       103|Electronics|       Phone|Completed|  East|    High|1500.0|    1300.0| user_3|
|       105|Electronics|      Tablet|Completed| North|  Medium|1100.0|     950.0| user_5|
|       101|Electronics|      Laptop|Completed| North|    High|1200.0|    1000.0| user_1|
|       102|   Clothing|      Jacket|  Pending| South|  Medium| 800.0|     700.0| user_2|
+----------+-----------+------------+---------+------+--------+------+----------+-------+
```

Q13: In Spark Architecture, what is the difference between **Client Mode** and **Cluster Mode**?

In Client Mode, the Driver runs on the machine from which the Spark application is submitted, while the Executors run inside the cluster. This mode is commonly used during development because the logs and results are easier to view directly on the client machine. However, the client must remain connected while the application is running.

In Cluster Mode, both the Driver and the Executors run inside the cluster. After submitting the application, the client machine does not need to remain connected. This makes Cluster Mode more suitable and reliable for production or long-running Spark jobs. The main difference between the two modes is therefore the location of the Driver.

Q14: Write a query to filter a dataset for rows where the `region` is 'North' OR the `priority` is 'High'.

```python
q14_result = df.filter(
    (col("region") == "North") |
    (col("priority") == "High")
)

q14_result.show()
```

```
+----------+-----------+------------+---------+------+--------+------+----------+-------+
|product_id|   category|product_name|   status|region|priority|amount|base_price|user_id|
+----------+-----------+------------+---------+------+--------+------+----------+-------+
|       101|Electronics|      Laptop|Completed| North|    High|1200.0|    1000.0| user_1|
|       103|Electronics|       Phone|Completed|  East|    High|1500.0|    1300.0| user_3|
|       105|Electronics|      Tablet|Completed| North|  Medium|1100.0|     950.0| user_5|
+----------+-----------+------------+---------+------+--------+------+----------+-------+
```

Q15: When exploring a dataset, why is it safer to use `.show(5)` instead of `.collect()` on a multi-terabyte dataset?

‘.show(5)’ is safer because it retrieves and displays only a small number of rows from the dataset. In contrast, ‘.collect()’ transfers every row from all the Executors to the Driver machine. For a multi-terabyte dataset, the Driver may not have enough memory to store all that data, which can lead to an out-of-memory error or cause the Spark application to crash. Therefore, ‘.show(5)’ should be used for quickly exploring large datasets, while ‘.collect()’ should only be used when the result is known to be very small.